

07 — Web Logs

Apache/IIS parsing and web shell detection for DFIR

Educational / Defensive DFIR Study Notes

LEARNING OBJECTIVES

- Parse Apache and IIS W3C fields correctly (especially changing #Fields)
- Filter logs by URI, status, IP, method, time, and request id
- Prove web shell execution via request + w3wp child process correlation
- Avoid the HTTP 200 = success trap

SCOPE REMINDER

- This material covers detection, artifacts, and incident response only.
- Attacker actions are phrased as 'adversaries may...' then pivot to evidence.
- Tool/command names appear as IOC strings / exam knowledge — not how-to guides.

Why web logs matter

Web servers are frequent initial access and persistence points. Logs show who requested what, when, and with which response code — but they do **not** by themselves prove server-side command execution. Always be ready to join logs to host process evidence.

Apache logs (common / combined)

Apache access logging is set with LogFormat + CustomLog. Two nicknamed presets show up constantly on exams and in the wild. Always confirm the nickname (or the literal format string) in `httpd.conf` / the vhost before you parse.

Common Log Format (CLF) — nickname common

```
LogFormat "%h %l %u %t \"%r\" %>s %b" common
CustomLog logs/access_log common
```

Combined Log Format — nickname combined

```
LogFormat "%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\"" combined
CustomLog logs/access_log combined
```

Combined = Common + Referer + User-Agent. Most modern sites use combined (or a custom string built from the same tokens).

Token	Meaning	DFIR use
%h	Remote host (client IP)	Source IP; watch reverse-proxy / X-Forwarded-For
%l	RFC 1413 identity	Almost always - (unused)
%u	Authenticated user	Non-dash = HTTP auth identity
%t	Request time [dd/Mon/yyyy:HH:mm:ss +/-zzzz]	Timeline; note offset vs host logs
%r	First request line (method URI protocol)	Split method / path / query for hunting
%>s	Final status code	Prefer %>s over %s for what the client got
%b	Response bytes (CLF: - if zero)	Odd sizes on shell paths; volume clues
%{Referer}i	Referer header (combined)	Navigation / CSRF / unexpected origin
%{User-Agent}i	User-Agent (combined)	Scanner vs browser vs automation

Sample combined line

```
203.0.113.50 - - [24/Sep/2026:01:15:42 +0800] "POST /uploads/x.php HTTP/1.1" 200 412 "-"
"curl/8.5.0"
```

- **%h** 203.0.113.50 · **%l/%u** both - · **%t** 24/Sep/2026:01:15:42 +0800
- **%r** POST /uploads/x.php HTTP/1.1 · **%>s** 200 · **%b** 412
- **Referer** - · **User-Agent** curl/8.5.0 (automation-like)
- Default paths: `logs/access_log` (or `access.log`) and `logs/error_log` — `error_log` is not CLF
- Hunt: rare URIs, `.php/.jsp/.aspx` under upload dirs, admin panels from odd IPs
- Status clusters: repeated 404 (scanning), sudden 500 (exploit fail), 200 on unusual paths
- Methods: unexpected PUT/MOVE/PROPFIND on IIS/WebDAV-enabled servers

IIS W3C logs — critical exam point

IIS W3C extended logs begin with directive lines. The **#Fields:** header defines column order — **and that order can change** across servers or after configuration updates. Never hard-code column positions from memory; always read `#Fields` for the file you are parsing.

Common field	Use in investigations
c-ip / s-ip	Client vs server address

Common field	Use in investigations
cs-method	GET/POST/etc.
cs-uri-stem / cs-uri-query	Path and query string — web shell params often here
sc-status	HTTP status returned
cs-user-agent	Scanner vs browser vs automation
cs(Referer)	Navigation path / CSRF clues
time / date	Align with host 4688 — watch time zones
s-port / cs-version	Service identification

Filtering strategy

- Narrow by time window of the alert
- Filter URI keywords: cmd, whoami, eval, .ashx, unusual upload paths
- Stack by IP and method; isolate rare combinations
- Use request id / correlation id if present to join to app traces
- Export candidates then pivot to the web server host

Web shells — proving execution

Adversaries may upload or write a web shell and invoke it over HTTP. A useful evidence chain on IIS:

- **Request** in IIS log to a suspicious ASPX/ASHX/PHP path (often POST)
- **Worker**: w3wp.exe (IIS application pool process) handles the request
- **Execution**: child processes such as cmd.exe or powershell.exe under w3wp — visible in 4688/Sysmon 1
- **On disk**: the shell file itself (hash, MFT create time near first request)

CRITICAL MISCONCEPTION

- HTTP 200 does NOT prove the attack succeeded — only that the server returned OK.
- A request in the log does NOT equal command execution.
- Prove execution with process creation (and preferably command line) tied to the worker process.
- Conversely, execution might occur even if you have incomplete logs — host evidence still matters.

How do I prove a web shell?

- Find first successful-looking request to the shell path in W3C/Apache logs
- On the server: locate file; hash; check write source (WebDAV, upload bug, RCE)
- Confirm w3wp (or apache/httpd/php-fpm) spawned shell/interpreter processes at that time
- Trace what commands ran; assess data access; contain (take app offline / remove shell / rotate)

Common exam traps

- Assuming fixed IIS column order without reading #Fields
- Equating HTTP 200 with compromise success
- Ignoring time zone skew between web logs and Windows event logs
- Hunting only .aspx while shells use .ashx, images with appended payloads, etc.
- Forgetting that reverse proxies change c-ip (use X-Forwarded-For if logged)

Lab takeaways

FROM LAB 4-1 — WEB ATTACKS / WEBSHELL

- Join WAF ↔ IIS on request ID (d4rid), not nearest timestamp; blocked probes may have zero IIS rows.
- Decode-then-match WAF order can miss encoded probes (control limitation).
- Prove execution: w3wp→cmd 4688 + matching request marker + app exit — not IIS 200 alone.
- Hayabusa rule: Suspicious Process By Web Server Process; watch writable uploads for .aspx.

QUICK REFERENCE

- Apache: common vs combined LogFormat tokens; confirm LogFormat
- Always parse IIS #Fields — order can change
- Filter: URI, status, IP, method, time, request id
- Web shell chain: request → w3wp.exe → cmd/powershell
- 200 ≠ success; request ≠ execution
- Join logs to 4688/Sysmon on the web server

Mini Self-Check

Answer these without looking back. Answers are on the next page.

- Q1.** Why must analysts read the IIS #Fields line every time?
- Q2.** What process typically hosts ASP.NET request execution on IIS?
- Q3.** What host evidence proves a web shell ran a command?
- Q4.** Why is HTTP 200 insufficient proof of a successful exploit?
- Q5.** Name four useful filter dimensions for web log triage.

Self-Check Answers

- A1. Because the field order can change across configurations; hard-coded positions mis-parse the log.*
- A2. w3wp.exe (IIS worker process for the application pool).*
- A3. Child process creation (e.g., cmd.exe or powershell.exe) under w3wp.exe correlated in time with the request, plus the shell file on disk.*
- A4. It only indicates the HTTP response code was OK; it does not prove malicious server-side execution.*
- A5. URI, status, IP, method (also timestamp and request id).*